```
foreach ($results as $node) {
    var_dump(trim($node->textContent));
}
```

```
Output:
string(27) "React with 10k dependencies"
string(29) "node.js with 10k dependencies"
string(11) "WIP CONCEPT"
string(16) "It's honest work"
string(29) "I was there... 3000 years ago"
```

Remember that the predicate is applied only to the localization step between the / separators, and you're perfectly fine to tack another step on afterwards. Don't forget you can use logical operations within predicates; here's how you can find all the child nodes of `projects` that have either `"php"` or `"book"` as one of their classes:

```
// ... $xml, $dom, and $xpath init as before
$results = $xpath->query(
    '//project[contains(@class, "php") or contains(@class, "book")]/*'
);
print "Found {$results->length} matching node(s)" . PHP_EOL;
foreach ($results as $node) {
    var_dump(trim($node->textContent));
}
```

```
Output:
Found 2 matching node(s)
string(16) "It's honest work"
string(49) "Deadlines make a whistling noise as they fly past"
```

Neat huh? One final curveball to throw at you: there's one special token in the XPath syntax that defies categorization because it belongs neither in axis, node test, nor predicate. It is possible to use multiple expressions with a union operator (which will combine the node list results of both expressions into a single set). The following examples return the same output as the previous...